A 文档样式

pyzdb 演示：从裸盘解析到 ZFS 对象模型

项目源码：https://github.com/zhengyp36/pyzdb

---

1. 演示目标

本演示的目标是说明：
如何仅依赖磁盘上的 ZFS 元数据，从裸盘开始，定位到一个 ZFS 存储池，并进入其内部对象模型。
更具体地说：

- 不依赖内核态 ZFS，不导入 pool
- 不使用 zdb、zpool 等系统工具
- 只通过磁盘上的 LABEL、uberblock、blkptr、dnode 等结构
- 在用户态还原 ZFS 的内部对象体系

pyzdb 的作用，是把这一条路径变成可以执行的程序化过程。

---

2. 实验环境与存储池结构

2.1 环境

- VirtualBox 7.0
- CentOS 8.3
- 6 块数据盘，每块 4GB

2.2 ZFS 池结构

```
zpool create poolx \
    raidz1 /dev/sdb /dev/sdc /dev/sdd \
    raidz1 /dev/sde /dev/sdf /dev/sdg
```

得到的拓扑为：

```
poolx
 ├── raidz1-0 : sdb, sdc, sdd
 └── raidz1-1 : sde, sdf, sdg
```

这是一个有 **两个 top-vdev** 的 pool，用来验证 nvlist 与 vdev 组织是否正确。

---

3. 从单盘读取 nvlist

ZFS 在每个磁盘的 LABEL 区域中保存 nvlist，用于描述该磁盘所属的 pool 以及它所在的 vdev 结构。

在 pyzdb 中：

```
from zdb import *

disk = Disk('/dev/sdb1')
raw_data = disk.read_nvpairs(label_index=0)
nvlist = NVList.from_bytes(raw_data)
```

得到的 nvlist（节选）：

```
name      : poolx
pool_guid: 0xf0a7ef4b5382af55

vdev_tree:
  type : raidz
  id   : 0
  children:
    [0] disk /dev/sdb1
    [1] disk /dev/sdc1
    [2] disk /dev/sdd1
```

这说明：

- 单个磁盘并不描述整个 pool
- 它只描述它所在的 **top-vdev**
- 通过所有磁盘的 nvlist，才能重建完整拓扑

---

4. 构建 vdev 树

pyzdb 使用 `VDevManager` 来扫描多个磁盘，并把它们的 nvlist 合并成一棵 vdev 树。

```
vdmgr = VDevManager()
vdmgr.scan(['/dev/sdb1'])
vdmgr.scan(['/dev/sdc1', '/dev/sdd1'])
vdmgr.scan(['/dev/sde1', '/dev/sdf1', '/dev/sdg1'])

vdmgr.ls()
```

输出：

```
poolx[GUID=0xf0a7ef4b5382af55]
  <raidz>[0]
    <disk>[0]/dev/sdb1
    <disk>[1]/dev/sdc1
    <disk>[2]/dev/sdd1
  <raidz>[1]
    <disk>[0]/dev/sde1
    <disk>[1]/dev/sdf1
    <disk>[2]/dev/sdg1
```

到这里，磁盘的层级关系已经恢复为 ZFS 的 vdev 结构。

5. 从 vdev 中选择 uberblock

ZFS 的入口不是某个固定地址，而是 **uberblock**。
每个磁盘的每个 LABEL 都包含一个 uberblock 数组。
每个 uberblock 记录了某一时刻整个 pool 的根指针。
uberblock 的大小由 ashift 决定：

```
ub_size = 2 ^ min(max(ashift, 10), 13)
```

在本环境中 ashift = 9，因此 ub_size = 1024 bytes。
遍历一个磁盘上的 uberblock：

```
def traverse_ub(disk_path):
    disk = Disk(disk_path)
    raw = memoryview(disk.read_uberblock(label_index=0))
    ub_sz = 1024

    ubs = []
    pos = 0
    while pos + ub_sz <= len(raw):
        try:
            ubs.append(UberBlock(raw[pos:pos+ub_sz]))
        except MagicError:
            pass
        pos += ub_sz
    return ubs

ubs = traverse_ub('/dev/sdb1')
[ub.ub_txg for ub in ubs]
```

示例结果：

```
[4, 5, 6, 393, 394, 3471, 3472, 3473, 3476, 3477, 3478, 410, 412]
```

选择 `txg` 最大的那个，即当前最可信的 pool 状态。

---

6. 从 uberblock 得到 rootbp

选中的 uberblock（示例）：

```
ub_txg = 3478
ub_rootbp = blkptr {
  dva:
    v1.of0005e.s2
    v0.of0005e.s2
    v1.o100002a.s2
  lsize = 4KB
  psize = 512B
  compress = LZ4
}
```

含义：

- meta-objset 被存了 3 份
- 分别位于两个 raidz vdev 中
- 通过 blkptr 即可定位并读取

到此为止，我们已经完成了：

> 从磁盘 → LABEL → uberblock → root blkptr

---

7. 读取 meta-objset（MOS）

接下来进入 ZFS 的对象世界。

```
rvd = vdmgr.lookup('poolx')
rvd.open()

reader = BlkPtrReader(rvd)
raw = reader.read(ub.ub_rootbp)

spa = Spa(rvd)
spa.reader = reader
mos = ObjSet(spa, ub.ub_rootbp)
```

`mos` 是 ZFS 的 **meta-objset**，即管理整个 pool 的对象集合。
其物理结构：

```
mos.phys.os_meta_dnode
```

这是一个 dnode，它的内容就是 **整个 MOS 的 dnode 表**。

---

8. dnode.read：进入 ZFS 的统一对象模型

到这一步，最重要的抽象已经建立：

> **一个 ZFS 对象，就是一个 dnode**
> **一个 dnode，通过 blkptr，指向它的数据**

无论是：

- objset

- dsl_dir
- dataset
- zap
- 文件
- 目录

它们在盘上的统一形式都是：

```
dnode → blkptr → （可能的间接块） → 数据块
```

pyzdb 在这一层实现了：

```
dnode.read(offset, size)
```

这个接口内部完成：

- blkid 计算
- 间接块遍历
- blkptr 解析
- raidz 映射
- 解压缩

使用者不需要再关心：

- 这个对象有几级间接块
- 它分布在哪个 vdev
- 是否被压缩

这意味着：

> 从 MOS 开始，可以用同一套机制，访问整个 ZFS 内部结构

---

9. 这个状态意味着什么

在本演示中，我们已经完成了 ZFS 最关键的三步：

1. **拓扑恢复**
   a. 从 nvlist 建立 vdev 树
2. **时间点选择**
   a. 从所有磁盘的 uberblock 中选出最优 txg
3. **对象世界入口**
   a. 从 uberblock 进入 MOS，并建立 dnode 读取机制

到这里，ZFS 的“磁盘格式”已经被还原成：

```
一个可以被遍历的对象空间
```

之后无论是：

- 定位某个 dataset
- 遍历某个目录
- 解析 zap
- 读取文件内容

都只是沿着：

```
ObjSet → dnode → blkptr → 数据
```

这一条统一路径向前走。

---

10. 结语

本演示的重点不在于“读出一个文件”，
而在于展示：

> **如何从裸盘出发，进入 ZFS 的对象模型。**

pyzdb 把这一条原本只存在于内核和 zdb 中的路径，
变成了一个可以在用户态执行、可调试、可扩展的系统。

附录 A：演示代码与操作步骤

本附录给出本文档中涉及的全部操作命令与 pyzdb 交互式演示代码，读者可以据此在本地复现实验过程。

A.1 创建存储池与测试文件

在一台 Linux 主机上，使用 6 块空盘创建 ZFS 存储池：

```
zpool create poolx \
    raidz1 /dev/sdb /dev/sdc /dev/sdd \
    raidz1 /dev/sde /dev/sdf /dev/sdg
```

在该池中创建一个文件系统，并写入测试文件：

```
zfs create poolx/fs1
echo "hello,world" > /poolx/fs1/test.txt
zpool export poolx
```

此时，数据与所有 ZFS 元数据已经写入磁盘。

A.2 构建并进入 pyzdb 环境

进入 pyzdb 目录并编译：

```
cd pyzdb
make
```

启动 Python 交互式环境：

```
python
```

导入 pyzdb：

```
from zdb import *
```

A.3 读取磁盘 LABEL 中的 nvlist

以其中一块磁盘为例（`/dev/sdb1`）：

```
disk = Disk('/dev/sdb1')
raw_data = disk.read_nvpairs(label_index=0)
nvlist = NVList.from_bytes(raw_data)
nvlist
```

该 nvlist 描述了此磁盘所属的 pool 与其所在的 vdev。

A.4 构建 vdev 树

扫描所有磁盘并构建 pool 拓扑：

```
vdmgr = VDevManager()

vdmgr.scan(['/dev/sdb1'])
vdmgr.scan(['/dev/sdc1', '/dev/sdd1'])
vdmgr.scan(['/dev/sde1', '/dev/sdf1', '/dev/sdg1'])

vdmgr.ls()
```

这一步把来自多个磁盘的 nvlist 合并为完整的 vdev 树。

A.5 遍历并选择 uberblock

定义遍历 uberblock 的辅助函数：

```
def traverse_ub(disk_path):
    disk = Disk(disk_path)
    raw = memoryview(disk.read_uberblock(label_index=0))
    ub_sz = 1024
    ubs = []
    pos = 0
    while pos + ub_sz <= len(raw):
        try:
            ubs.append(UberBlock(raw[pos:pos+ub_sz]))
        except MagicError:
            pass
        pos += ub_sz
    return ubs
```

定义选择最大 txg 的函数：

```
def sel_ub(ubs):
    ans = ubs[0]
    for ub in ubs[1:]:
        if ub.ub_txg > ans.ub_txg:
            ans = ub
    return ans
```

在某个磁盘上遍历并选择 uberblock：

```
ubs = traverse_ub('/dev/sdb1')
[ub.ub_txg for ub in ubs]

ub = sel_ub(ubs)
ub
ub.ub_rootbp
```

`ub.ub_rootbp` 即 meta-objset 的根 blkptr。

A.6 打开 pool 并读取 meta-objset

查找并打开 pool 的 root vdev：

```
rvd = vdmgr.lookup('poolx')
rvd.open()
```

构造 SPA 与 blkptr 读取器：

```
spa = Spa(rvd)
spa.reader = BlkPtrReader(rvd)
```

根据 uberblock 的 rootbp 构造 meta-objset：

```
mos = ObjSet(spa, ub.ub_rootbp)
```

A.7 从 meta-dnode 读取一个 dnode

meta-objset 的 `metadn` 是整个 MOS 的 dnode 表。
首先计算单个 dnode 的大小：

```
dnsz = DNodePhys.sizeof()
```

读取第 1 个 dnode（示例）：

```
raw = mos.metadn.read(1 * dnsz, dnsz)
DNodePhys(raw)
```

此时，已经完成了从：

```
磁盘 → uberblock → rootbp → meta-objset → meta-dnode → 普通 dnode
```

的完整路径。

A.8 本附录在整体演示中的位置

这一组命令与代码，覆盖了本文档中最关键的技术路径：

- nvlist 解析
- vdev 树构建
- uberblock 选择
- blkptr 读取
- dnode 抽象的建立

在此基础上，后续所有 ZFS 对象（objset、zap、文件、目录）的访问，都统一建立在：

```
dnode.read(offset, size)
```

这一接口之上。